



Operador *in* e Caracteres

Pertinência: operador in

O operador booleano **in** verifica se uma dada string pertence (ou está presente ou contido) em outra string. Analogamente: o **in** verifica se uma string é uma **substring** de uma outra string (*sequência $x \subset$ sequência y* ?)

"gosto" in "Eu gosto de programar!"	—————→	True
"odeio" in "Eu gosto de programar!"	—————→	False
"y" in "Python"	—————→	True
"i" in "Python"	—————→	False

Testa a **ausência** de uma substring usando **not**.

"odeio" **not in** "Eu gosto de programar!" —————→ **True**

Obs. A string vazia é uma substring de qualquer *string*.

"" **in** "Python" —————→ **True**

Caracteres e strings

- **Quem são os caracteres?**

Letras: 'a', 'd', 'F', 'G'

Símbolos: '%', '@', ':', '?'

Dígitos: '0', '3', '6', '9'

Símbolo Especiais: '\n', '\t', '\b', '\r'

“símbolos que normalmente encontra-se num teclado de um computador”.

- Em Python, um caractere é uma string de comprimento 1.

COMO O CARACTERE É REPRESENTADO INTERNAMENTE?

Para serem processados pelo computador, os caracteres devem ser representados por números.

- **tabela de códigos: nº ↔ caracter**
define correspondência entre caracteres e códigos numéricos

Exemplos: ASCII: 256 caracteres diferentes (Python 2.x)

Unicode UTF-8: 65536 caracteres diferentes(no mínimo) (Python 3.x)

ASCII vs UTF-8

Most Significant Character									
Hex	0	1	2	3	4	5	6	7	
0	NUL	DLE	Space	0	@	P	`	p	Least Significant Character
1	SOH	DC1	!	1	A	Q	a	q	
2	STX	DC2	"	2	B	R	b	r	
3	ETX	DC3	#	3	C	S	c	s	
4	EOT	DC4	\$	4	D	T	d	t	
5	ENQ	NAK	%	5	E	U	e	u	
6	ACK	SYN	&	6	F	V	f	v	
7	Bell	ETB	'	7	G	W	g	w	
8	BS	CAN	(	8	H	X	h	x	
9	HT	EM	)	9	I	Y	i	y	
A	LF	SUB	*	:	J	Z	j	z	
B	VT	ESC	+	;	K	[	k	{	
C	FF	FS	,	<	L	\	l		
D	CR	GS	-	=	M	]	m	}	
E	SO	RS	.	>	N	^	n	~	
F	SI	US	/	?	O	_	o	DEL	

U+0000 000 001 002 003 004 005 006 007 008 009 00A 00B 00C 00D 00E 00F

0	NUL	DLE	SP	0	@	P	`	p	PAD	DCS	NS SP	°	À	Đ	à	đ
1	SOH	DC1	!	1	A	Q	a	q	HOP	PU1	ı	±	Á	Ñ	á	ñ
2	STX	DC2	"	2	B	R	b	r	BPH	PU2	ϕ	²	Â	Ò	â	ò
3	ETX	DC3	#	3	C	S	c	s	NBH	STS	£	³	Ã	Ó	ã	ó
4	EOT	DC4	\$	4	D	T	d	t	IND	SCS	¥	´	Ä	Ô	ä	ô
5	ENQ	NAK	%	5	E	U	e	u	NEL	MW	¥	µ	Å	Õ	å	õ
6	ACK	SYN	&	6	F	V	f	v	SSR	SPR	ı	¶	Æ	Ö	æ	ö
7	BEL	ETB	'	7	G	W	g	w	ESR	EPB	§	·	Ç	×	ç	÷
8	BS	CAN	(	8	H	X	h	x	HTS	SOS	ˆ	˘	È	Ø	è	ø
9	HT	EM	)	9	I	Y	i	y	HTJ	SGC	©	¹	É	Ù	é	ù
A	LF	SUB	*	:	J	Z	j	z	VTJ	SCI	ª	º	Ê	Ú	ê	ú
B	VT	ESC	+	;	K	[	k	{	PLD	CSI	«	»	Ë	Û	ë	û
C	FF	FS	,	<	L	\	l		PLU	ST	¬	¼	Ï	Ü	ï	ü
D	CR	GS	-	=	M	]	m	}	RI	OSC	SHV	½	Í	Ý	í	ý
E	SO	RS	.	>	N	^	n	~	SS2	PM	®	¾	Î	Þ	î	þ
F	SI	US	/	?	O	_	o	DEL	SS3	APC	ˉ	¿	Ï	ß	ï	ÿ

Funções ord e chr

A função `ord` converte um caractere para o código numérico correspondente na codificação Unicode.

- ✓ nessa codificação, os códigos das letras seguem a “ordem alfabética” e, por convenção, os números das minúsculas são maiores do que os das maiúsculas.

<code>print(ord('A'))</code>	→	65
<code>print(ord('B'))</code>	→	66
<code>print(ord('B')-ord('A'))</code>	→	1
<code>print(ord('a'))</code>	→	97

A função `chr` é a inversa de `ord`. Ela recebe um código numérico e retorna o caractere correspondente na codificação Unicode.

<code>print(chr(65))</code>	→	A
<code>print(chr(66))</code>	→	B

Caracteres: mão na massa! (1/2)

1. Escreva uma função que recebe um caractere e, caso o caractere seja uma letra minúscula, retorna a letra maiúscula correspondente. Caso contrário, retorna o caractere.
2. Escreva uma função que recebe um caractere e, caso o caractere seja uma letra maiúscula, retorna a letra minúscula correspondente. Caso contrário, retorna o caractere.

Caracteres: mão na massa! (2/2)

3. Escreva uma função que recebe uma string `X` e retorne uma string `Y` criada do seguinte modo:
 - O caractere `'-'` pertence a `X` – retorne a segunda metade de `X` + `'&'` + primeira metade de `X`;
 - O caractere `'-'` não pertence a `X` – retorne a primeira metade de `X` + `'-'` + segunda metade de `X`
 - O 1º caractere de `Y` é uma letra minúscula - torne-a maiúscula;
 - O último caractere de `Y` é uma letra maiúscula - torne-a minúscula;
4. Escreva uma função, chamada **ehDoEstado**, que receba a placa de um carro e um dos prefixos de um estado. Esta função retorna a parte numérica, se a placa for do estado, ou 0, caso contrário.
 - ✓ Por exemplo, as placas do Rio variam de `KMF0001` a `LVE9999`. Logo, a chamada `ehDoEstado('LRS3232', 'LRS')` deve retornar `3232`

Possíveis soluções: 1 e 2

1)

```
def paraMaiuscula(c):  
    if c >= 'a' and c <= 'z':  
        return chr(ord('A') + (ord(c) - ord('a')))  
    else:  
        return c
```

→ Soluções usam a ordem relativa das letras no alfabeto, mesmo desconhecendo os códigos numéricos!

2)

```
def paraMinuscula(c):  
    if c >= 'A' and c <= 'Z':  
        return chr(ord('a') + (ord(c) - ord('A')))  
    else:  
        return c
```

Possível solução: 3

3)

```
def novaString(strX):  
    meio = len(strX)//2  
    if '-' in strX:  
        strY= strX[meio:] + '&' +  
        strX[:meio]  
    else:  
        strY= strX[:meio] + '-' +  
        strX[meio:]  
  
    prim = paraMaiuscula(strY[0])  
    ult = paraMinuscula(strY[-1])  
    return prim+ strY[1:-1]+ ult
```

Possível solução: 4

4)

```
def ehDoEstado(placa, prefixo):  
    if prefixo in placa:  
        return int(placa[3:])  
    else:  
        return 0
```